

ВЛОЖЕННЫЕ КЛАССЫ И ИНТЕРФЕЙСЫ-GENERIC- ИСКЛЮЧЕНИЯ

Современные Технологии Программирования

Ахмад Али

Вложенные Классы (Нестатические)

- Нестатический вложенный класс (non-static inner class) — это класс, объявленный внутри другого класса и связанный с экземпляром внешнего класса.
- Имеет доступ ко всем членам внешнего класса, включая `private`.
- Создаётся через объект внешнего класса.
- Не может содержать статических членов

Вложенные Классы (Статические)

- Статический вложенный класс (static inner class) — это класс, объявленный внутри другого класса с модификатором `static`.
- Не имеет доступа к нестатическим членам внешнего класса
- Создаётся без объекта внешнего класса
- Может содержать статические и нестатические члены

Вложенные Интерфейсы

- Вложенный интерфейс — это интерфейс, объявленный внутри другого класса или интерфейса.
- По умолчанию он является `static`, даже если это явно не указано.
- Если интерфейс вложен в класс, он может иметь модификаторы доступа `public`, `protected` или `private`.
- Если интерфейс вложен в другой интерфейс, он может быть только `public` и всегда `static`.
- Вложенные интерфейсы используются для определения контракта, тесно связанного с внешним классом или интерфейсом.
- Они могут быть реализованы внешним или любым другим классом.

Вложенные Классы и Интерфейсы

- Вложенные классы (как статические, так и нестатические) могут наследоваться от других классов, не связанных с внешним классом.
- Они также могут реализовывать интерфейсы, включая вложенные.

Вложенные Классы и Интерфейсы



Generics

- Обобщения (Generics) — это механизм, который позволяет писать универсальный код, работающий с разными типами данных, не указывая конкретные типы заранее.
- Позволяют использовать переменные типа в определении классов, интерфейсов и методов.
- Основная цель — обеспечение безопасности типов во время компиляции.

Generics

- Классы и интерфейсы могут быть параметризованы одним или несколькими типами.
- Методы могут иметь собственные параметры типа, независимо от класса.
- Поддерживаются ограниченные параметры (extends) для указания допустимых типов.
- Обобщения повышают модульность, обеспечивая универсальные алгоритмы.
- Используются в коллекциях (List, Map, Set) и пользовательских структурах данных.

Generics

- Обобщения существуют только на этапе компиляции.
- После компиляции происходит стирание типов (type erasure).
- Компилятор заменяет параметры типа их верхней границей (Object или extends).
- В результирующем байткоде не сохраняется информация о параметрах типа.
- Это обеспечивает обратную совместимость с более старыми версиями Java.
- Из-за стирания невозможно использовать instanceof с параметрами типа.
- Также нельзя создавать массивы обобщённых типов напрямую.
- Подклассы могут расширять обобщённые классы
 - с конкретизацией (class StringChild extends Parent<String>)
 - с сохранением параметров типа (class Child<T> extends Parent<T>).

Generics

- Джокеры (?) позволяют указывать неизвестные или гибкие типы.
 - ? extends T — любой подтип T.
 - ? super T — любой супертип T.
- Без ограничений (?) — универсальная ссылка на любой параметр типа.
- Используются в API коллекций и при универсальной обработке данных.
- Требуют аккуратного подхода, чтобы не нарушить безопасность типов.

Generics



Исключения

- Исключения — это события, нарушающие нормальный поток выполнения программы.
- Все исключения наследуются от `Throwable`.
- Основные категории:
 - `Error` — критические ошибки среды выполнения, не подлежат обработке (`OutOfMemoryError`, `StackOverflowError`).
 - `Exception` — ошибки, которые могут быть обработаны программой.
- Исключения делятся на две группы:
 - `Checked` (проверяемые) — требуют обязательной обработки.
 - `Unchecked` (непроверяемые) — могут не обрабатываться явно.

Исключения

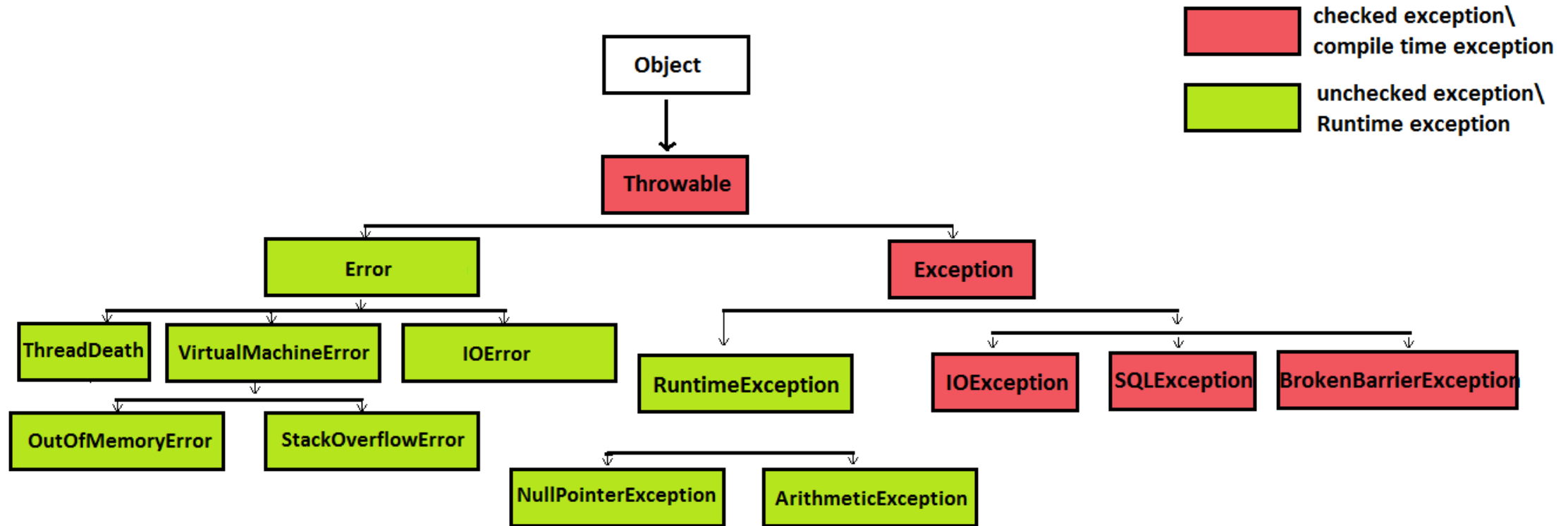
Проверяемые исключения (Checked exceptions):

- Подклассы Exception, но не RuntimeException.
- Обязательно обрабатываются с помощью try-catch или объявляются через throws.
- Представляют ожидаемые, внешние ошибки (например, ошибки ввода-вывода, работы с файлами, базой данных и т.д.).
- Примеры: IOException, SQLException, ParseException.

Проверяемые исключения (Unchecked exceptions):

- Подклассы RuntimeException.
- Не требуют обязательной обработки.
- Представляют ошибки программирования (например, доступ к null, выход за границы массива).
- Примеры: NullPointerException, IndexOutOfBoundsException, IllegalArgumentException.

Исключения



Исключения

- Ключевые слова Java, связанные с исключениями:
 - try — определяет блок кода, где может возникнуть исключение.
 - catch — обрабатывает конкретный тип исключения.
 - finally — выполняется всегда, независимо от наличия исключения.
 - throw — инициирует исключение вручную.
 - throws — указывает, что метод может выбрасывать исключения.
- Один try может сопровождаться несколькими catch.
- Блок finally обычно используется для закрытия ресурсов.
- Пользовательские исключения создаются путем наследования от классов исключений.

Исключения

- try-with-resources (автоматическое закрытие ресурсов)- специальная форма try, введённая в Java 7.
- Используется для автоматического закрытия ресурсов, реализующих интерфейс AutoCloseable.
- Позволяет избежать утечек памяти и ресурсоёмких ошибок.
- Не требует явного вызова close() — ресурсы закрываются автоматически после выполнения блока.
- Может сопровождаться блоками catch и finally, как обычный try.
- Наиболее часто применяется при работе с файлами, потоками, соединениями с БД и т. п.
- Рекомендуется использовать вместо ручного закрытия ресурсов в finally.

Исключения

